Kernel

$$k(x_i, x_j) = \sigma_f^2 \exp\{-\frac{1}{2l^2}(x_i - x_j)^T(x_i - x_j)\}$$

```
import numpy as np
def kernel(X1, X2, l=1.0, sigma_f = 1):
    sqdist = np.sum(X1**2, 1).reshape(-1, 1)+ np.sum(X2**2, 1) - 2 * np.dot(X1, X2.T)
    return sigma_f**2 * np.exp(-0.5 * sqdist / l**2)
matrix1 = np.array([[1, 2], [3, 4]])
matrix2 = np.array([[1, 2], [3, 4], [5, 6]])
print(kernel(matrix1, matrix2))
```

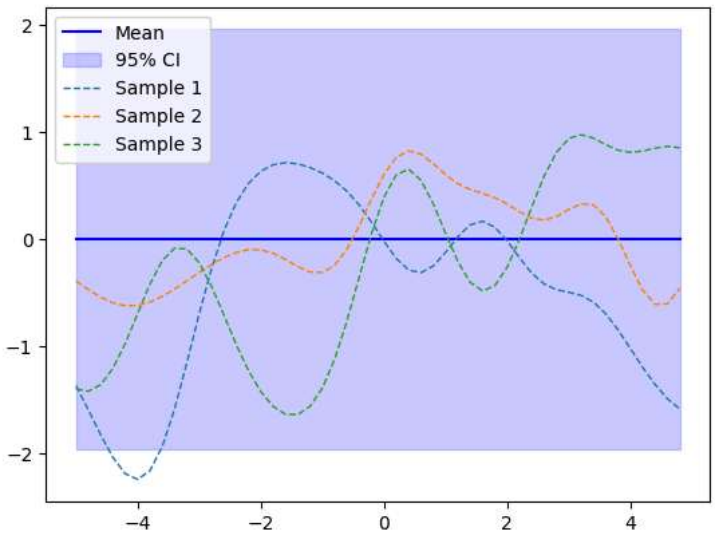
```
[[1.00000000e+00 1.83156389e-02 1.12535175e-07]
 [1.83156389e-02 1.00000000e+00 1.83156389e-02]]
```

prior

```
# X = np.arange(-5, 5, 0.2).reshape(-1,1)
# X = np.arange(-5, 5, 0.2)
# print(X.shape)
# X = X.reshape(-1, 1)
# print(X.shape)
```

```
(50,)
(50, 1)
```

```
%matplotlib inline
import matplotlib.pyplot as plt

def plot_gp(mu, cov, X, X_train=None, Y_train=None, samples=[]):
        X = X.ravel()
        mu = mu.ravel()
        std = np.sqrt(np.diag(cov))                    # 1 standard deviation
        uncertainty = 1.96 * std                           # 95% confidence interval

        plt.plot(X, mu, 'b-', label='Mean')
        plt.fill_between(X, mu - uncertainty, mu + uncertainty,
                                 alpha=0.2, color='blue', label='95% CI')

        for i, sample in enumerate(samples):
                plt.plot(X, sample, lw=1, ls='--', label=f'Sample {i+1}')

        if X_train is not None:
                plt.scatter(X_train, Y_train, c='red', zorder=5, label='Training points')

        plt.legend()


X = np.arange(-5, 5, 0.2).reshape(-1,1)
mu = np.zeros(X.shape)
cov = kernel(X, X)
samples = np.random.multivariate_normal(mu.ravel(), cov, 3) # generate 3 samples from mu=[0,...,0], cov = (50,50), each
plot_gp(mu, cov, X, samples=samples)
```



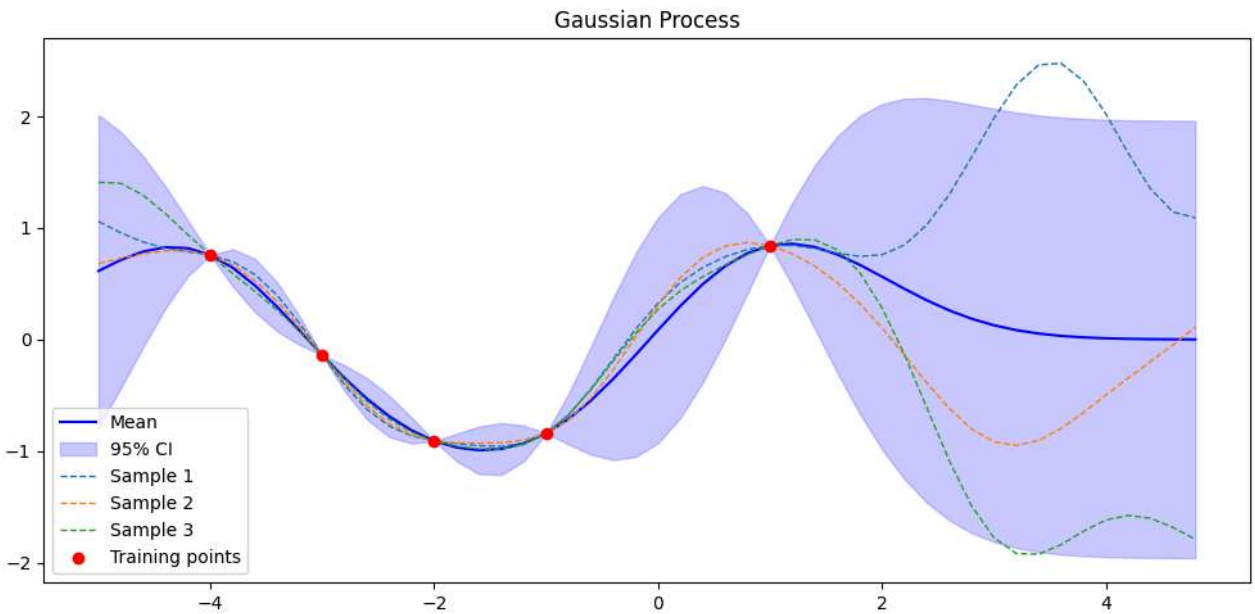
posterior predict from noise-free training data

```
from numpy.linalg import inv
def posterior(X_s, X_train, Y_train, l=1.0, sigma_f = 1.0, sigma_y=0):
    K = kernel(X_train, X_train, l, sigma_f) + sigma_y**2 * np.eye(len(X_train))
    K_s = kernel(X_train, X_s, l, sigma_f)
    K_ss = kernel(X_s, X_s, l,sigma_f)
    K_inv = inv(K)

    mu_s = K_s.T.dot(K_inv).dot(Y_train)
    cov_s = K_ss - K_s.T.dot(K_inv).dot(K_s)
    return mu_s, cov_s
```

```
X_train = np.array([-4, -3, -2, -1, 1]).reshape(-1, 1)
Y_train = np.sin(X_train)
mu_s, cov_s = posterior(X, X_train, Y_train)
print(mu_s.shape)
print(mu_s.ravel().shape)
```

```
(50, 1)
(50,)
```

```
X_train = np.array([-4, -3, -2, -1, 1]).reshape(-1, 1)
Y_train = np.sin(X_train)
mu_s, cov_s = posterior(X, X_train, Y_train)
samples = np.random.multivariate_normal(mu_s.ravel(), cov_s, 3)
plot_gp(mu_s, cov_s, X, X_train=X_train, Y_train=Y_train, samples=samples)
```

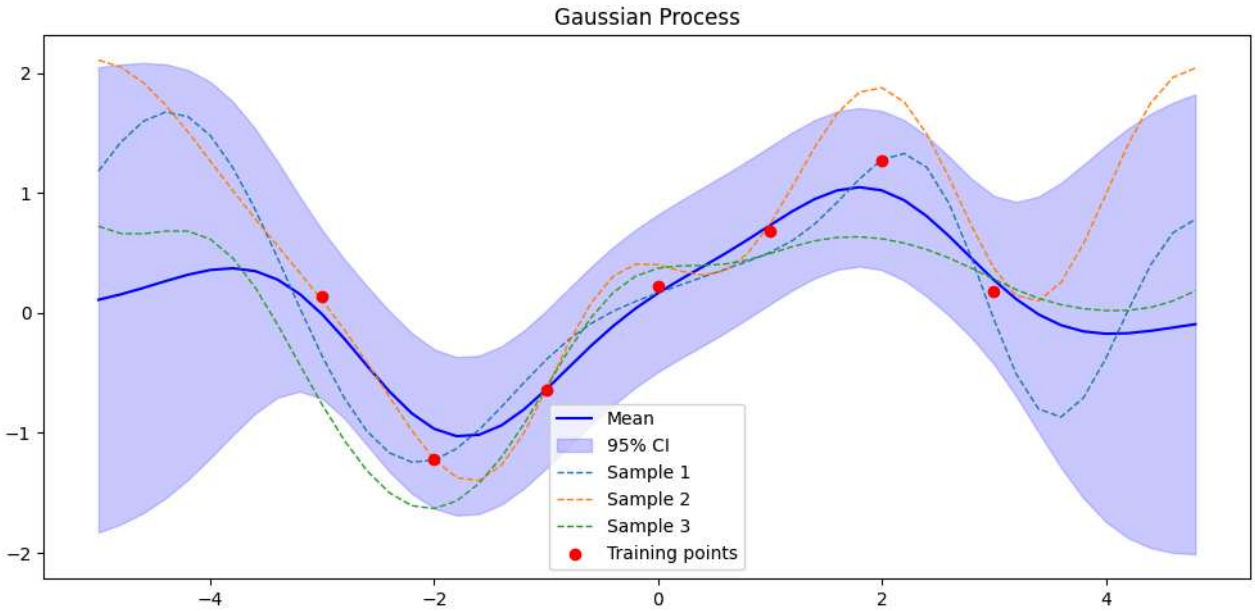


prediction from noisy data

```
shape = (3, 4)

# np.random.randn(shape)        # ❌ error
np.random.randn(*shape)       # ✅ same as np.random.randn(3, 4)
```

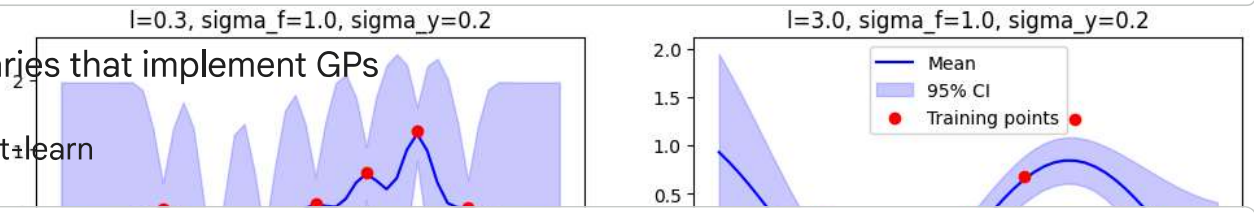
```
array([[-0.07933836, -0.53969124, -1.47080759, -0.2879662 ],
       [-0.79734906, -0.5878273 ,  0.52760154, -0.74542572],
       [ 2.40103782,  0.72878557, -1.66756997,  0.16594977]])
```

```
noise = 0.4
X_train = np.arange(-3, 4, 1).reshape(-1, 1)
Y_train = np.sin(X_train) + noise * np.random.randn(*X_train.shape)
mu_s, cov_s = posterior(X, X_train, Y_train, sigma_y=noise) # set noise exactly to be sigma_y
samples = np.random.multivariate_normal(mu_s.ravel(), cov_s, 3)
plot_gp(mu_s, cov_s, X, X_train, Y_train, samples = samples)
```



effect of kernel parameters and noise parameter

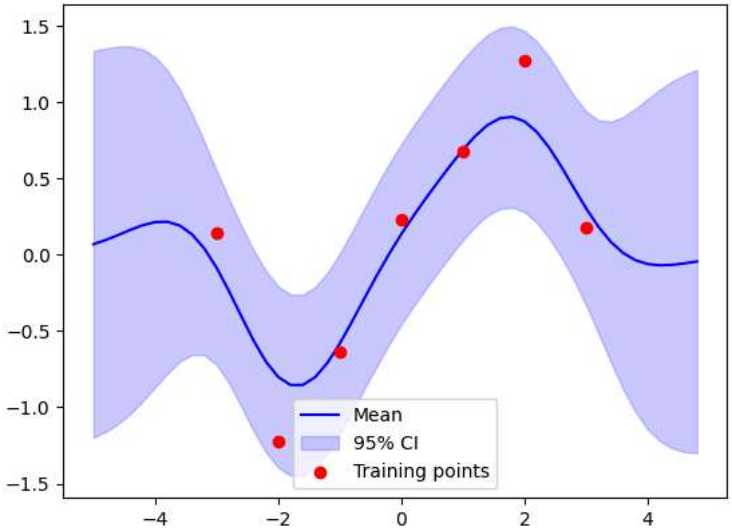
```
params = [
        (0.3, 1.0, 0.2),
        (3.0, 1.0, 0.2)
]

plt.figure(figsize=(12, 5))
for i, (l, sigma_f, sigma_y) in enumerate(params):
    mu_s, cov_s = posterior(X, X_train, Y_train, l=l, sigma_f=sigma_f, sigma_y=sigma_y)
    plt.subplot(3, 2, i+1)
    plt.subplots_adjust(top=2)
    plt.title(f'l={l}, sigma_f={sigma_f}, sigma_y={sigma_y}')
    plot_gp(mu_s, cov_s, X, X_train, Y_train)
```



libraries that implement GPs

Scikit-learn

```
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import ConstantKernel, RBF

rbf = ConstantKernel(1.0) * RBF(length_scale=1.0) # this defines the kernel where constantkernel(1.0) is the signal vari
gpr = GaussianProcessRegressor(kernel=rbf, alpha=noise**2) # alpha is the sigma_y^2 that means noisy data with sigam_y^2
gpr.fit(X_train, Y_train)
mu_s, cov_s = gpr.predict(X, return_cov=True)
l = gpr.kernel_.k2.get_params()['length_scale']
sigma_f = np.sqrt(gpr.kernel_.k1.get_params()['constant_value'])
plot_gp(mu_s, cov_s, X, X_train, Y_train)
```



GPy

```
import GPy

rbf = GPy.kern.RBF(inpt_dim=1, variance=1, lengthscale=1)
gpr = GPy.models.GPRegression(X_train, Y_train, rbf)
gpr.Gaussian_noise.variance = noise**2
gpr.Gaussian_noise.variance.fix()

gpr.optimize()
l = gpr.rbf.lengthscae.values[0]
sigma_f = np.sqrt(gpr.rbf.variance.values[0])
gpr.plot()
```

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
/tmp/ipykernel_77852/3858463443.py in <cell line: 0>()
----> 1 import GPy
      2
      3 rbf = GPy.kern.RBF(inpt_dim=1, variance=1, lengthscale=1)
      4 gpr = GPy.models.GPRegression(X_train, Y_train, rbf)
      5 gpr.Gaussian_noise.variance = noise**2

ModuleNotFoundError: No module named 'GPy'

---------------------------------------------------------------------------
NOTE: If your import is failing due to a missing package, you can
manually install dependencies using either !pip or !apt.

To view examples of installing some common dependencies, click the
"Open Examples" button below.
---------------------------------------------------------------------------
```

OPEN EXAMPLES